

Session 1 Date: April 29, 2026 (2 hours)

Objective:

In this session, practice BSP with all serial, parallel runners and BFS.

Attempts made:

The BspAlgorithm base class and the serial runner was the first runner that I was able to implement. With the serial runner, I was able to implement my runner pretty easily since I would just loop HasWork() and process the vertices, then call a PostRound().

The parallel runner was a lot harder to get going. The initial parallel runner was calculating the chunks to process by trying to use the following:

```
uint32_t chunk = g.num_vertices / nthreads;
```

The way I calculated chunks in my implementation, it would cause the implementation to drop the last vertices of my graph, because `g.num_vertices` wasn't evenly divided by `nthreads`.

In parallel BFS implementation, I also was reading and writing in `Parallel Process()` of the BSP algorithm and used the same `dist` array, as it trying to achieve a read, write, read, write, read, parallelism, and reading the parallel edges instead of the parallel in-edges, which is basically the opposite of the BSP pull model.

Solutions/Workarounds:

The following ceiling division was used with a fixed chunk size:

```
uint32_t chunk = (g.num_vertices + nthreads - 1) / nthreads;
```

Solving BFS required a method known as double buffering, where `Process` would write to `dist` and read from `Process` from `prev`, and `dist` would be updated as follows:

```
int32_t candidate = prev[u] + 1;
```

```
if (best == -1 || candidate < best)
```

```
best = candidate;
```

```
dist[v] = best;
```

The line `PostRound` would make the execution of the round write to `prev`, ensuring the next round would read from a write stable snapshot of the array, fixing the race condition.

I was able to fix the logic of the implementation by scanning the in-edges, which correctly implemented the logic of the BSP model.

Commits:

The BSP serial and parallel runners and both implementations of BSP BFS were included alongside implementations of serial versus parallel.

Experiments/Analysis:

I executed the test on the disconnected graph consisting of (0 1 1 / 1 2 1 / 3 4 1). For the point of interest (3, 4), the graph traversal algorithm is expected to return -1, as those vertices are unreachable from the source vertex (0).

Also, I validated graph traversal as well on the graph consisting of (0 1 1 / 1 2 1 / 2 0 1). As a cyclic graph, I validated that the algorithm terminated, preventing overlap. Both serial and parallel BFS remained in alignment.

Learning outcome:

In the BSP model, double buffering is a must. If double buffering is not implemented, it is the case that the threads perform a read on values that are written in the same time frame in a write operation by another thread, thus, making those reads non-deterministic.

Parallel safety is ensured by static partitioning. Since it is guaranteed that a single thread operates on a single vertex (i.e. no two threads can write to the same vertex), and it is also guaranteed that in the shared memory at the beginning of the round 'r', nothing is read, nothing is written, the same threads operate in the same memory, and nothing is read or written in the shared memory at the beginning of the round 'r'.